

Reading Support: Computing Admissible Strings I

Companion to Week 9

Cem Bozsahin

LING487 · Fall 2026

Normal forms support uniform methods

Supports: Normal form trades surface variety for a uniform method

A context-free grammar is in Chomsky normal form when every nonempty rule has the form $A \rightarrow BC$ or $A \rightarrow a$. Greibach normal form requires each nonempty right-hand side to begin with a terminal.

Every context-free language has an equivalent grammar in these forms, subject to the usual treatment of ϵ . The transformation preserves the language, not automatically the original analysis.

MOLR §4.5

Binarization inserts method-specific structure

Supports: Binarization creates the spans CKY needs

Replacing $A \rightarrow BCD$ by $A \rightarrow BX$ and $X \rightarrow CD$ ensures that each nonlexical rule joins two constituents. Another binarization could group B and C first.

Both may preserve the terminal strings. The added nonterminal and its grouping require separate justification if the resulting tree is interpreted linguistically.

MOLR §4.5

CKY stores categories by substring span

Supports: CKY turns parsing into a triangular record of spans

For a grammar in Chomsky normal form, the CKY chart begins with categories that rewrite as each terminal. A longer cell receives A when some split divides the span into adjacent cells containing B and C , with $A \rightarrow BC$ in the grammar.

The string belongs to the language exactly when the start symbol appears in the cell covering the complete string.

MOLR §4.5

Every longer chart cell requires all possible splits

Supports: A four-word CKY chart records all binary combinations

A span from position i through j has split points after $i, i+1, \dots, j-1$. CKY checks every pair of categories licensed across those divisions.

The chart avoids recomputing a substring because all categories already established for that span remain available to every larger cell that contains it.

MOLR §4.5

Recognition can preserve several parses compactly

Supports: Ambiguity survives as multiple derivations in one cell

One category may enter a cell through several rule applications or split points. Backpointers retain those alternatives without copying the category itself.

For an ambiguous expression grammar, the complete-span cell can therefore contain E once while pointing to the different bracketings that derive it. Recognition and recovery of structure remain distinguishable tasks.

MOLR §4.5

A PDA configuration includes unread input and stack

Supports: A pushdown automaton adds unbounded stack memory

A pushdown automaton $M = (Q, \Sigma, \Gamma, \Delta, s, F)$ has finite state control and a stack alphabet. A configuration $[q, \alpha, \gamma]$ records the current state, unread input, and stack content.

A transition may consume a symbol or ϵ , change state, remove a stack symbol, and add stack material. Acceptance requires a stated relation among final state, exhausted input, and stack.

MOLR §5.1

The stack carries a context-free dependency

Supports: The stack preserves a dependency after its trigger disappears

For $a^n b^n$, the machine can push one marker for each a and remove one for each following b . The stack retains the effect of an earlier symbol after the input head has moved beyond it.

Finite state alone cannot distinguish arbitrarily many unmatched a 's. The stack supplies the required unbounded but disciplined memory.

MOLR §5.1

Top-down and bottom-up parsers reverse the direction of search

Supports: Top-down parsing predicts; bottom-up parsing recognizes

A top-down parser begins with the start symbol and expands categories until their terminal expectations match the input. A bottom-up parser begins with input symbols and reduces recognized right-hand sides to their left-hand categories.

The first can pursue grammatical predictions unsupported by the input. The second can construct locally licensed fragments that never join into a complete analysis.

MOLR §§5.2–5.3

Left recursion can prevent a naive method from progressing

Supports: Left recursion is a methodological problem, not an illicit structure

With $A \rightarrow A\alpha$, a naive top-down parser may expand A again before consuming any input. Rewriting the grammar into a right-recursive form can preserve its stringset while allowing the method to advance.

The difficulty belongs to the interaction of grammar and parsing strategy. It does not show that the intended left-branching structure is linguistically impossible.

MOLR §5.2